

Here is a simple, complete Native Query program (minimum files) in [Spring Boot](#).
You can copy this and run directly.

✓ 1. Main Class

```

</> Java
package com.example.nativequery;

import org.springframework.boot.SpringApplication;
import org.springframework.boot.autoconfigure.SpringBootApplication;

@SpringBootApplication
public class NativeQueryApplication {

    public static void main(String[] args) {
        SpringApplication.run(NativeQueryApplication.class, args);
    }

}

```

✓ 2. Entity Class

```

</> Java
package com.example.nativequery.entity;

import jakarta.persistence.*;

@Entity
@Table(name = "patient")
public class Patient {

    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;

    private String name;
    private int age;

    public Patient() {}

    public Patient(String name, int age) {
        this.name = name;
        this.age = age;
    }

    public Long getId() { return id; }
    public String getName() { return name; }
    public int getAge() { return age; }

    public void setName(String name) { this.name = name; }
    public void setAge(int age) { this.age = age; }

}

```

✓ 3. Repository (Native Query)

```
</> Java

package com.example.nativequery.repository;

import com.example.nativequery.entity.Patient;
import org.springframework.data.jpa.repository.*;
import org.springframework.data.repository.query.Param;

import java.util.List;

public interface PatientRepository extends JpaRepository<Patient, Long> {

    // Native Query
    @Query(value = "SELECT * FROM patient WHERE age > :age", nativeQuery = true)
    List<Patient> findPatientsAboveAge(@Param("age") int age);
}
```

✓ 4. Controller

◆ Get Plus ×

```
</> Java

package com.example.nativequery.controller;

import com.example.nativequery.entity.Patient;
import com.example.nativequery.repository.PatientRepository;

import org.springframework.web.bind.annotation.*;

import java.util.List;

@RestController
@RequestMapping("/api/patient")
public class PatientController {

    private final PatientRepository repository;

    public PatientController(PatientRepository repository) {
        this.repository = repository;
    }

    // Test API
    @GetMapping("/check")
    public String check() {
        return "Native Query API Working";
    }

    // Insert Data
    @PostMapping("/add")
    public String addPatient(@RequestBody Patient patient) {
        repository.save(patient);
        return "Patient added!";
    }

    // Native Query API
    @GetMapping("/above/{age}")
    public List<Patient> getPatientsAboveAge(@PathVariable int age) {
        return repository.findPatientsAboveAge(age);
    }
}
```

✓ ◆ 5. application.properties

🔗 properties

server.port=8082

spring.datasource.url=jdbc:mysql://localhost:3306/testdb
spring.datasource.username=root
spring.datasource.password=1234

spring.jpa.hibernate.ddl-auto=update
spring.jpa.show-sql=true

✓ ◆ 6. pom.xml

🔗 XML

```
<project xmlns="http://maven.apache.org/POM/4.0.0">  
  <modelVersion>4.0.0</modelVersion>  
  
  <groupId>com.example</groupId>  
  <artifactId>nativequery</artifactId>  
  <version>0.0.1-SNAPSHOT</version>  
  
  <parent>  
    <groupId>org.springframework.boot</groupId>  
    <artifactId>spring-boot-starter-parent</artifactId>  
    <version>3.2.0</version>  
  </parent>  
  
  <dependencies>  
  
    <!-- REST -->  
    <dependency>  
      <groupId>org.springframework.boot</groupId>  
      <artifactId>spring-boot-starter-web</artifactId>
```

✓ ◆ Project Structure

```
com.example.nativequery  
├─ NativeQueryApplication.java  
├─ controller  
│   └─ PatientController.java  
├─ entity  
│   └─ Patient.java  
├─ repository  
│   └─ PatientRepository.java
```